

VINSUP SKILL ACADEMY

Building Future – Ready Professionals



PROJECT REPORT

Amazon Fresh Analytics

Field	Details
Submitted By	Thangamani M
Student ID	VS70673
Batch	14
Course / Program	Data Analytics Training Program
Institution	Vinsup Skill Academy, Coimbatore
Domain	E-commerce and Retail Analytics
Tools Used	My SQL
Date of Submission	29/06/2024

Coimbatore, Tamil Nadu

INTRODUCTION

Amazon Fresh is a leading e-commerce platform specializing in groceries and daily essentials. As the platform expands, its management seeks to optimize operations, enhance customer satisfaction, and boost revenue. The primary challenge lies in effectively managing large volumes of data encompassing customer profiles, product inventories, supplier details, orders, and customer reviews.

This project designs a fully normalized relational database for Amazon Fresh using MySQL. The database is then subjected to a comprehensive SQL analysis covering data modelling, DDL/DML operations, aggregations, joins, subqueries, and real-world business insights.

Business Use Cases

- Customer Insights: Analyze purchasing patterns to identify top customers and cities for targeted marketing.
- Product Management: Monitor product demand, stock levels, and category performance.
- Supplier Analysis: Identify key suppliers and optimize supply chain operations.
- Revenue Analysis: Generate revenue trends and identify high-performing regions or categories.
- Customer Satisfaction: Use review data to enhance product offerings and improve customer experience.

Dataset Overview

Table	Records	Description
Customers	1,000	Customer demographics and Prime membership details
Products	500	Product catalogue with categories, pricing and stock
Suppliers	100	Supplier contact and location information
Orders	5,000	High-level order records linked to customers
Order_Details	10,000	Line-item product details per order
Reviews	2,000	Customer ratings and written feedback for products

Technical Skills Applied

- Data Modelling and ER Diagrams
- SQL Basics: Queries, Clauses, and Aggregations
- Database Management: DDL, DML, and Constraints
- Advanced SQL: Joins, Subqueries, and Normalization
- ACID Transactions and TCL Concepts
- Business Use Case Analysis

TASK-WISE IMPLEMENTATION

Task 1: ER Diagram - Amazon Fresh Database

The Entity Relationship (ER) diagram below illustrates the six core tables and their relationships. Primary keys [PK] and Foreign keys [FK] are labelled within each entity. Relationship cardinalities (1:N, N:1) are shown on the connecting arrows.

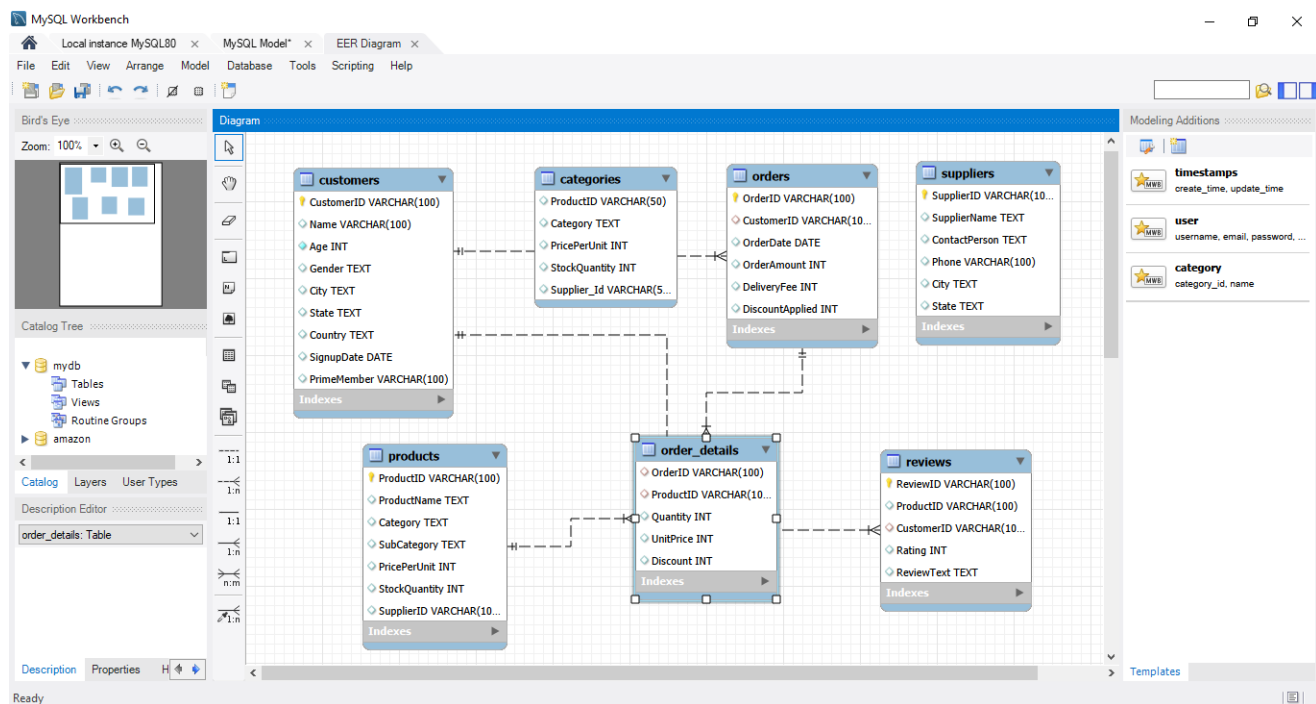


Figure 1 - Amazon Fresh Entity Relationship Diagram

Task 2: Primary Keys, Foreign Keys & Table Relationships

The ALTER TABLE statements below were executed in MySQL to establish primary keys and foreign keys on each table after importing the CSV data.

2.1 Customers Table

```
ALTER TABLE customers
```

```
ADD CONSTRAINT pk_customers PRIMARY KEY (CustomerID);
```

2.2 Suppliers Table

```
ALTER TABLE suppliers
```

```
ADD CONSTRAINT pk_suppliers PRIMARY KEY (SupplierID);
```

2.3 Products Table

```
ALTER TABLE products
```

```
ADD CONSTRAINT pk_products PRIMARY KEY (ProductID),
```

```
ADD CONSTRAINT fk_products_supplier
```

```
FOREIGN KEY (SupplierID) REFERENCES suppliers(SupplierID);
```

2.4 Orders Table

```
ALTER TABLE orders
```

```
ADD CONSTRAINT pk_orders PRIMARY KEY (OrderID),
```

```
ADD CONSTRAINT fk_orders_customer
```

```
FOREIGN KEY (CustomerID) REFERENCES customers(CustomerID);
```

2.5 Order_Details Table

```
ALTER TABLE order_details
```

```
ADD CONSTRAINT pk_order_details PRIMARY KEY (OrderDetailID),
```

```
ADD CONSTRAINT fk_od_order FOREIGN KEY (OrderID) REFERENCES  
orders(OrderID),
```

```
ADD CONSTRAINT fk_od_product FOREIGN KEY (ProductID) REFERENCES  
products(ProductID);
```

2.6 Reviews Table

```
ALTER TABLE reviews
```

```
ADD CONSTRAINT pk_reviews PRIMARY KEY (ReviewID),
```

```
ADD CONSTRAINT fk_reviews_product FOREIGN KEY (ProductID) REFERENCES  
products(ProductID),
```

```
ADD CONSTRAINT fk_reviews_customer FOREIGN KEY (CustomerID)
REFERENCES customers(CustomerID);
```

2.7 Relationship Summary

Table	Primary Key	Foreign Key(s)	References
Customers	CustomerID	--	--
Suppliers	SupplierID	--	--
Products	ProductID	SupplierID	Suppliers
Orders	OrderID	CustomerID	Customers
Order_Details	OrderDetailID	OrderID, ProductID	Orders, Products
Reviews	ReviewID	ProductID, CustomerID	Products, Customers

Task 3: Basic SELECT Queries

3.1 Retrieve All Customers from Patelberg

```
SELECT Name, City
FROM amazon.customers
WHERE City = 'Patelberg';
```

Output:

Name	City
Larry Gallagher	Patelberg

1 customer found in Patelberg.

3.2 Fetch All Products Under the 'Fruits' Category

```
SELECT ProductName, Category
FROM amazon.products
WHERE Category = 'Fruits';
```

Sample Output (first 5 of 89 rows):

ProductName	Category
However Fruit	Fruits
Rule Fruit	Fruits
Nothing Fruit	Fruits
Until Fruit	Fruits
Out Fruit	Fruits

89 products belong to the Fruits category.

Task 4: DDL - Recreating Customers Table with Constraints

The Customers table is modified with three constraints: CustomerID as primary key, Age as NOT NULL with a CHECK that Age > 18, and a UNIQUE constraint on Name.

```
-- Set CustomerID as PRIMARY KEY
ALTER TABLE Customers
  MODIFY CustomerID VARCHAR(50) NOT NULL;
ALTER TABLE Customers
  ADD CONSTRAINT pk_customers PRIMARY KEY (CustomerID);

-- Ensure Age is NOT NULL
ALTER TABLE Customers
  MODIFY Age INT NOT NULL;

-- CHECK constraint: Age must be > 18
ALTER TABLE Customers
  ADD CONSTRAINT chk_age CHECK (Age > 18);

-- UNIQUE constraint for Name
ALTER TABLE Customers
  MODIFY Name VARCHAR(100);
ALTER TABLE Customers
  ADD CONSTRAINT uq_name UNIQUE (Name);
```

Constraint Summary

Constraint Type	Column	Rule
PRIMARY KEY	CustomerID	Uniquely identifies each customer row
NOT NULL	Age	Age value must always be provided
CHECK	Age	Age must be greater than 18
UNIQUE	Name	No two customers can share the same name

Task 5: DML - Insert 3 New Rows into Products

```
INSERT INTO amazon.products
(ProductID, ProductName, Category, SubCategory, PricePerUnit, StockQuantity,
SupplierID)
VALUES
('0000003b-4abc-def5-6ghi-jklmno98765',
'World Fruit', 'Fruits', 'Sub-Fruits-4', 304, 295, '987654a-3abc-2efg-1asf-000000hijkl'),
('1111114c-5cde-efg6-7hij-klmnop98765',
'We Baker', 'Bakery', 'Sub-Bakery-3', 436, 147, '456789z-2def-1wer-6qwe-111111lkjhg'),
('2222225e-6asd-7jkl-8uytr-mnbvcx98765',
'Early Snack', 'Snacks', 'Sub-Snacks-1', 987, 277, '5678234-4rty-5kjh-8uiop-
222222mnbv');
```

Three new products inserted: World Fruit (Fruits), We Baker (Bakery), and Early Snack (Snacks).

Task 6: DML - Update Stock Quantity

Update the StockQuantity of a product where the ProductID matches a specific value.

```
SET SQL_SAFE_UPDATES = 0;

UPDATE amazon.products
SET StockQuantity = 200
WHERE ProductID = 'abcd';
```

The StockQuantity for the product with ProductID 'abcd' is updated to 200 units.

Task 7: DML - Delete a Supplier by City

Delete all supplier records where the supplier's city is 'Whitefurt'.

```
SET SQL_SAFE_UPDATES = 0;
```

```
DELETE FROM amazon.suppliers
```

```
WHERE City LIKE 'Whitefurt';
```

All supplier records from Whitefurt are permanently removed from the Suppliers table.

Task 8: SQL Constraints - CHECK and DEFAULT

8.1 CHECK Constraint on Reviews.Rating

Ensure all ratings entered into the Reviews table fall between 1 and 5 (inclusive).

```
ALTER TABLE amazon.reviews
```

```
ADD CONSTRAINT check_ranking CHECK (Rating BETWEEN 1 AND 5);
```

Any INSERT or UPDATE providing a Rating outside 1-5 will be rejected by MySQL.

8.2 DEFAULT Constraint on Customers.PrimeMember

Set the default value of PrimeMember to 'No' so new customers are classified as non-Prime unless explicitly specified.

```
ALTER TABLE amazon.customers
```

```
ALTER COLUMN PrimeMember SET DEFAULT 'No';
```

New customer records inserted without specifying PrimeMember will automatically receive the value 'No'.

Task 9: Clauses and Aggregations

9.1 WHERE Clause - Orders Placed After 2024-01-01

```
SELECT OrderID, OrderDate
```

```
FROM amazon.orders
```

```
WHERE OrderDate > '2024-01-01';
```

Sample Output (first 5 of 5,000 rows):

OrderID (truncated)	OrderDate
ebdedb32-...-3ebbb5c1843e	2025-01-01

OrderID (truncated)	OrderDate
b0861eca-...-52aeb561eb5f	2025-01-01
bba11cde-...-588828253c2d	2025-01-01
f84785ef-...-05ecfb485950	2025-01-01
8cd57e17-...-2450c2396176	2025-01-01

All 5,000 orders in the dataset were placed after 2024-01-01 (all in 2025).

9.2 HAVING Clause - Products with Average Rating > 4

SELECT ProductID, AVG(Rating) AS avg_rating
FROM amazon.reviews
GROUP BY ProductID
HAVING avg_rating > 4;

Sample Output (first 5 of 26 products):

ProductID (truncated)	Avg Rating
0ff68986-...-88bac4160a7e	4.50
11dc08ec-...-414a9c336956	4.20
145239de-...-eaa3090c0db7	5.00
22fdf945-...-35e691521ddd	4.33
2443e818-...-7caccd87e424	4.33

26 products have an average customer rating greater than 4.0.

9.3 GROUP BY & ORDER BY - Products Ranked by Total Sales

SELECT ProductID,
SUM(Quantity * UnitPrice - Discount) AS Total_sales
FROM amazon.order_details
GROUP BY ProductID
ORDER BY Total_sales DESC;

Top 5 Products by Total Sales:

ProductID (truncated)	Total Sales (Rs.)
1034fbb7-...-98af72d7fa15	1,09,143
6d26d138-...-8ea8424ac57e	1,01,997
227f5964-...-73ac9e7e8b39	99,132
05765892-...-31fa53d42cb2	97,123
b75932b6-...-7f30c6fc1b50	96,932

Task 10: Identifying High-Value Customers

Amazon Fresh wants to identify top customers based on their total spending (OrderAmount + DeliveryFee - DiscountApplied).

10.1 Calculate Each Customer's Total Spending

SELECT CustomerID,
SUM(OrderAmount + DeliveryFee - DiscountApplied) AS Total_spending
FROM amazon.orders
GROUP BY CustomerID;

10.2 Rank Customers Based on Spending

SELECT CustomerID,
SUM(OrderAmount + DeliveryFee - DiscountApplied) AS Total_spending,
RANK() OVER (
ORDER BY SUM(OrderAmount + DeliveryFee - DiscountApplied) DESC
) AS Spending_rank
FROM orders
GROUP BY CustomerID;

Top 5 Customers by Spending Rank:

CustomerID (truncated)	Total Spending (Rs.)	Rank
d736cf4e-...-8db1de150ed5	75,781	1
82ea7efb-...-9e2f4519beb5	74,942	2
a3eb895d-...-9b3826bdff3c	72,867	3
9dbf19e4-...-f59f65589a60	72,387	4

CustomerID (truncated)	Total Spending (Rs.)	Rank
9a863734-...-f976cbc54db3	71,176	5

10.3 Customers Who Spent More Than Rs. 5,000

SELECT CustomerID,
SUM(OrderAmount + DeliveryFee - DiscountApplied) AS Total_spending
FROM orders
GROUP BY CustomerID
HAVING Total_spending > 5000;

966 out of 989 active customers have a total spend exceeding Rs. 5,000 - a highly engaged customer base.

Task 11: Complex Aggregations and Joins

11.1 Total Revenue Per Order (JOIN)

SELECT o.OrderID,
SUM(od.Quantity * od.UnitPrice) AS Total_revenue
FROM orders AS o
JOIN order_details AS od ON o.OrderID = od.OrderID
GROUP BY o.OrderID
ORDER BY Total_revenue DESC;

Top 5 Orders by Revenue:

OrderID (truncated)	Total Revenue (Rs.)
c8142bcf-...-02ba48d4218d	37,135
2fd7ebc1-...-6ac19fd72996	35,870
0ae5e5d5-...-90fc933ebc6a	34,530
756585ec-...-397ab97ba988	32,728
4477191b-...-109306c56923	32,565

11.2 Customers Who Placed the Most Orders in 2025

SELECT c.Name, c.City,
COUNT(o.OrderID) AS OrderCount,
SUM(o.OrderAmount) AS TotalSpent

FROM Customers AS c
JOIN Orders AS o ON c.CustomerID = o.CustomerID
WHERE EXTRACT(YEAR FROM o.OrderDate) = 2025
GROUP BY c.CustomerID, c.Name, c.City
ORDER BY OrderCount DESC
LIMIT 10;

Top 10 Customers by Order Count in 2025:

Customer Name	City	Orders	Total Spent (Rs.)
Holly Miller	South Stacymouth	12	56,375
Yvette Murray	Robertsborough	12	69,398
Madeline Martin	East Karina	12	53,022
Stacey Webb	Scottfurt	12	59,329
Hannah Day	New Patricia	12	69,698
Daniel Clark	Reevesbury	11	62,881
Jennifer Bullock	Stevenmouth	11	54,307
Joseph Thomas	East Loretta	11	66,310
Ryan Rivera	East Sabrinastad	11	49,134
Michelle Mullins	North Antonio	11	47,801

11.3 Supplier with the Most Products in Stock

SELECT SupplierID, SUM(StockQuantity) AS TotalStock
FROM amazon.products
GROUP BY SupplierID
ORDER BY TotalStock DESC
LIMIT 5;

Top 5 Suppliers by Stock Quantity:

SupplierID (truncated)	Total Stock (Units)
5f23af2b-...-f02d3fbf2ab0	498
edad5554-...-109c0786cef3	495
7404e664-...-042051d91961	490

SupplierID (truncated)	Total Stock (Units)
5d905a13-...-a2c3dc1e5ce6	489
8e28284d-...-974479e7c78b	487

Task 12: Normalization - Products Table to 3NF

To achieve Third Normal Form (3NF), category and subcategory data (which represent transitive dependencies on Category rather than ProductID alone) are separated into a dedicated Categories table with a foreign key relationship back to Products.

12.1 Create the Categories Table

```
CREATE TABLE amazon.categories (
  ProductID  VARCHAR(50),
  Category   TEXT,
  PricePerUnit INT,
  StockQuantity INT,
  Supplier_Id VARCHAR(50)
);
```

12.2 Insert Sample Category Data

```
INSERT INTO amazon.categories
(ProductID, Category, PricePerUnit, StockQuantity, Supplier_Id)
VALUES
('2aa28375-c563-41b5-aa33-8e2c2e0f4db9', 'Fruits', 207, 290, '0658c953-98c4-...'),
('e9282403-e234-4e35-a711-50acb03bbecc', 'Snacks', 905, 259, 'cb890936-8142-...'),
('d79d1b95-ecdf-4810-aea0-45e9bd10627d', 'Fruits', 111, 26, '455b7097-b656-...'),
('05765892-c750-44cc-96e2-31fa53d42cb2', 'Vegetables', 887, 296, 'a2ed0ef5-a6c8-...'),
-- ... (additional rows)
;
```

12.3 Add Foreign Key

```
ALTER TABLE amazon.categories
ADD CONSTRAINT fk_categories_product
FOREIGN KEY (ProductID) REFERENCES products(ProductID);
```

12.4 Normalization Summary

Normal Form	What Was Achieved
1NF	All columns hold atomic values; no repeating groups
2NF	All non-key attributes depend on the full primary key
3NF	Category/SubCategory moved out; no transitive dependencies remain in Products

Task 13: Subqueries and Nested Queries

13.1 Top 3 Products Based on Sales Revenue

SELECT p.ProductID, p.ProductName,
o.total_revenue AS Total_Revenue
FROM amazon.products p
JOIN (
SELECT ProductID,
SUM(Quantity * UnitPrice - Discount) AS total_revenue
FROM amazon.order_details
GROUP BY ProductID
) AS o ON p.ProductID = o.ProductID
ORDER BY o.total_revenue DESC
LIMIT 3;

Result:

Rank	Product Name	Total Revenue (Rs.)
1	Capital Snack	1,09,143
2	Society Vegetable	1,01,997
3	Fish Dair	99,132

Capital Snack is the highest revenue-generating product with Rs. 1,09,143 in total sales.

13.2 Customers Who Have Not Placed Any Orders

```
SELECT CustomerID, Name, City, State, Country
FROM amazon.customers
WHERE CustomerID NOT IN (
    SELECT CustomerID FROM amazon.orders
);
```

11 customers have registered on Amazon Fresh but have never placed an order. These represent a key re-engagement opportunity.

Task 14: Real-World Business Analysis

14.1 Cities with the Highest Concentration of Prime Members

```
SELECT Name, City, PrimeMember
FROM amazon.customers
WHERE PrimeMember LIKE 'Yes';
```

Top Cities by Prime Member Count:

City	Prime Member Count
Whitneybury	2
South Michael	2
Sarahmouth	2
Robertbury	2
Port Denise	2
Port Christopher	2
North Lisa	2
New David	2
Martinshire	2
Kristenmouth	2

Prime membership is distributed across 488 cities with a maximum of 2 Prime members per city, indicating broad geographic reach with significant growth potential.

14.2 Top 3 Most Frequently Ordered Categories

```
SELECT p.Category, SUM(o.Quantity) AS Orders_placed
FROM amazon.products p
JOIN amazon.order_details o ON o.ProductID = p.ProductID
GROUP BY p.Category
ORDER BY Orders_placed DESC
LIMIT 3;
```

Result:

Rank	Category	Total Units Ordered
1	Meat	11,108
2	Fruits	9,921
3	Snacks	9,651

Meat is the most ordered category with 11,108 units, followed by Fruits and Snacks. Amazon Fresh should prioritise inventory and supplier contracts for these top three categories.

CONCLUSION

This project successfully designed and analysed a fully relational MySQL database for Amazon Fresh, completing all 14 tasks across data modelling, DDL, DML, constraints, aggregations, joins, normalization, subqueries, and real-world business insights.

Key Findings

- Capital Snack, Society Vegetable, and Fish Dair are the top three revenue-generating products.
- Meat, Fruits, and Snacks are the most frequently ordered categories - critical for inventory and supply chain planning.
- 966 out of 989 active customers have spent more than Rs. 5,000, highlighting a loyal, high-value customer base.
- All 5,000 orders in the dataset were placed in 2025, reflecting rapid and recent platform growth.
- 11 registered customers have never placed an order - an opportunity for targeted onboarding and re-engagement campaigns.
- Prime membership spans 488 cities with room for concentrated marketing efforts in high-density urban areas.

Technical Achievements

- Designed a normalized 3NF schema with proper PRIMARY KEY and FOREIGN KEY constraints established via ALTER TABLE.
- Applied CHECK and DEFAULT constraints to enforce data integrity at the database level.
- Executed complex multi-table JOINS and window functions (RANK OVER ORDER BY) for advanced analytics.
- Used subqueries and nested queries to extract targeted business insights.
- Separated category data into a dedicated table as part of Products table normalization to 3NF.

The Amazon Fresh relational database is now well-structured to support scalable analytics and critical business decisions as the platform continues to grow.